

30 CÂU HỎI TRẮC NGHIỆM & ĐÁP ÁN CHI TIẾT TF-IDF & INFORMATION RETRIEVAL

Hệ Thống Trợ Giúp Học Tập

Ngày 17 tháng 9 năm 2026

PHẦN 1: NỘI DUNG CÂU HỎI

- Câu 1:** Nếu mở rộng từ Unigram sang Bigram `ngram_range=(1,2)` trong `TfidfVectorizer`, điều gì sẽ xảy ra với ma trận TF-IDF?
- Tất cả giá trị IDF của cụm 2 từ đều bằng 0.
 - Kích thước từ điển $|V|$ bùng nổ tăng mạnh, ma trận trở nên thưa hơn nữa, nhưng bất được cụm 2 từ liền kề (ngữ cảnh ngắn).
 - Ma trận chuyển hoàn toàn từ thưa sang đặc.
 - Kích thước từ điển $|V|$ giảm đi một nửa.
- Câu 2:** Trong xử lý tiếng Việt, nếu không sử dụng thư viện tách từ chuyên dụng (như `pyvi` hay `vncorenlp`) mà chỉ phân tách theo khoảng trắng, từ phức “bệnh nhân” sẽ bị tách thành hai token đơn [bệnh, nhân]. Việc này gây ra tác hại gì cho bài toán tìm kiếm?
- Làm cho thuật toán không chạy được do lỗi cú pháp.
 - Làm sai lệch hoàn toàn ngữ nghĩa, biến một thuật ngữ y tế thành hai từ đơn lẻ có nghĩa rộng và nhiều cao (“bệnh” và “nhân” – người).
 - Giảm kích thước từ điển đi 10 lần.
 - Làm tất cả giá trị TF-IDF bằng 1.0.
- Câu 3:** Giả sử đối với truy vấn Q , tập dữ liệu có tổng cộng 4 tài liệu thực sự liên quan (Ground-truth relevant docs = 4). Hệ thống tìm kiếm trả về danh sách Top-5, trong đó chỉ có 2 tài liệu thực sự liên quan. Chỉ số Recall@5 đạt bao nhiêu?
- 0.40 (tức 40%)
 - 0.25 (tức 25%)
 - 0.50 (tức 50%)
 - 0.80 (tức 80%)
- Câu 4:** Tại sao danh sách từ vựng `vocabulary_` thu được từ `CountVectorizer` của Scikit-Learn luôn được sắp xếp theo thứ tự bảng chữ cái (alphabetical order)?
- Bắt buộc theo quy định của chuẩn Python 3.
 - Đảm bảo tính nhất quán của chỉ số cột (column index) trong ma trận vector hóa trên các lần chạy khác nhau.
 - Giúp thuật toán Cosine tính toán nhanh hơn 100 lần.
 - Để tự động xóa các từ đứng ở cuối bảng chữ cái.
- Câu 5:** Trong Python khi xây dựng ma trận TF-IDF cho 30.000 văn bản, tại sao người ta bắt buộc dùng định dạng `scipy.sparse.csr_matrix` thay vì mảng 2D Numpy thông thường (`numpy.ndarray`)?

- a. Do yêu cầu bắt buộc của hàm `TfidfVectorizer` để tránh lỗi tràn số nguyên.
- b. Vì CSR matrix tự động loại bỏ các từ OOV (Out Of Vocabulary).
- c. Tiết kiệm RAM bằng cách chỉ lưu các giá trị khác 0 và tối ưu hóa các phép toán nhân ma trận theo hàng (Compressed Sparse Row).
- d. Vì Numpy không hỗ trợ phép tính Cosine Similarity.

Câu 6: Người dùng tìm kiếm truy vấn “COVID-19 vaccine”. Tài liệu D chứa cụm từ “coronavirus vaccination”. Tại sao TF-IDF không xếp D vào kết quả mặc dù hai khái niệm này tương đồng cao?

- a. TF-IDF thất bại do gặp cả hai vấn đề: Khác biệt từ vựng (Vocabulary Mismatch) và thiếu chuẩn hóa gốc từ (Stemming/Lemmatization).
- b. Do từ “vaccine” có IDF bằng 0.
- c. Do ma trận thưa bị lỗi khi tính toán số thực.
- d. Do tài liệu D quá dài làm điểm Cosine bằng 0.

Câu 7: Hệ thống tìm kiếm đánh giá trên 3 truy vấn Q_1, Q_2, Q_3 . Vị trí của tài liệu liên quan đầu tiên thu được lần lượt là: Q_1 ở rank 1, Q_2 không tìm thấy tài liệu liên quan nào trong Top-10 (coi như $r_q \rightarrow \infty$), Q_3 ở rank 4. Chỉ số MRR của hệ thống là bao nhiêu?

- a. 0.333
- b. 0.250
- c. 0.417
- d. 0.625

Câu 8: Cho vector $x = [1, 2, 3]$ và $y = [2, 4, 6]$. Giá trị $\cos(x, y)$ là bao nhiêu?

- a. 1.0
- b. 0.0
- c. 0.5
- d. 2.0

Câu 9: Trong đoạn code tính IDF cho toàn bộ corpus gồm N tài liệu (docs là danh sách các danh sách token):

```
df = Counter()
for d in docs:
    df.update(set(d))
```

Tại sao phải dùng `set(d)` thay vì đưa trực tiếp `d` vào `df.update()`?

- a. Để loại bỏ tất cả các từ dài hơn 10 ký tự.
- b. Để mỗi từ xuất hiện nhiều lần trong cùng 1 tài liệu chỉ được đếm df đúng 1 lần cho tài liệu đó.
- c. Để sắp xếp các từ theo thứ tự bảng chữ cái.
- d. Để chuyển các từ thành số nguyên tự động.

Câu 10: Mẫu số trong công thức tính chỉ số Recall@K

$$R@K = \frac{\text{Số tài liệu liên quan lấy được}}{\text{Mẫu số}}$$

luôn là đại lượng nào?

- a. Số lượng từ trong câu truy vấn.

- b. Tổng số tài liệu N của tập corpus.
- c. Tổng số tài liệu thực sự liên quan hiện có trong tập dữ liệu (Ground-truth relevant documents).
- d. Số cố định K .

Câu 11: Giá trị lớn nhất mà một phần tử $tf(t, d)$ (tần suất tương đối) có thể đạt được trong một tài liệu là bao nhiêu?

- a. Bằng số lượng tài liệu N .
- b. 100.0
- c. 1.0 (khi tài liệu chỉ chứa duy nhất một từ lặp đi lặp lại).
- d. Vô cùng (∞).

Câu 12: Tính Cosine Similarity giữa hai vector $x = [2, 0, 1, 2]$ và $y = [1, 3, 0, 1]$. Làm tròn 3 chữ số thập phân.

- a. 0.500
- b. 0.667
- c. 0.402
- d. 0.816

Câu 13: Trong `TfidfTransformer`, nếu đặt `smooth_idf=False`, công thức IDF được sử dụng sẽ là:

- a. $idf(t) = \log(N/df(t)) + 1$
- b. $idf(t) = \log((1 + N)/df(t))$
- c. $idf(t) = 1/df(t)$
- d. $idf(t) = N/df(t)$

Câu 14: Mục đích toán học chính của việc thêm số 1 vào mẫu số và tử số trong công thức Smoothing IDF là gì?

$$idf(t) = \log \left(\frac{1 + N}{1 + df(t)} \right) + 1$$

- a. Triệt tiêu các từ xuất hiện quá nhiều lần trong tài liệu.
- b. Ép tất cả các giá trị TF-IDF phải là số nguyên.
- c. Làm cho ma trận TF-IDF luôn có tổng theo cột bằng 0.
- d. Tránh lỗi chia cho 0 (Division by zero) và đảm bảo $idf \geq 1$ không bị nhận giá trị 0 hoặc âm.

Câu 15: Trong bài toán Document Search trên tập 30K văn bản, nếu người dùng tăng K (số lượng kết quả trả về trong Top-K) từ $K = 5$ lên $K = 50$, xu hướng thay đổi trung bình của Precision@K và Recall@K sẽ như thế nào?

- a. Recall@K có xu hướng tăng hoặc giữ nguyên; Precision@K có xu hướng giảm.
- b. Cả Precision@K và Recall@K đều tăng mạnh.
- c. Cả hai chỉ số đều giảm do pha loãng mật độ từ vựng.
- d. Precision@K tăng, còn Recall@K giảm.

Câu 16: Tóm tắt bản chất cốt lõi của đại lượng $TF-IDF(t, d, D)$ trong một câu ngắn gọn nhất:

- a. Đo tốc độ xử lý chuỗi văn bản của máy tính.
- b. Đo xác suất từ đó bị gõ sai chính tả.
- c. Đo độ dài của tài liệu so với từ điển.

- d. Đo lường tầm quan trọng của một từ đối với một tài liệu cụ thể trong bối cảnh của một tập hợp tài liệu.

Câu 17: So sánh hai kỹ thuật chuẩn hóa từ: **Stemming** (Porter) và **Lemmatization** (WordNet). Phát biểu nào sau đây đúng nhất về tác động của chúng lên ma trận TF-IDF?

- a. Stemming luôn mang lại điểm MRR cao hơn Lemmatization trong mọi trường hợp.
- b. Stemming làm tăng kích thước từ điển, còn Lemmatization làm giảm kích thước từ điển.
- c. Stemming dùng quy tắc cắt gọt chuỗi (heuristics) nhanh hơn nhưng dễ tạo ra từ không có nghĩa; Lemmatization dùng từ điển ngữ pháp chính xác hơn nhưng tốn thời gian tính toán hơn.
- d. Lemmatization loại bỏ hoàn toàn hiện tượng ma trận thưa.

Câu 18: Khi thực hiện Punctuation Normalization bằng cách thay thế tất cả ký tự không phải chữ cái/số thành khoảng trắng trên chuỗi:

`state-of-the-art model cost $1,000`

Chuỗi token thu được (phân tách theo khoảng trắng) chứa bao nhiêu token?

- a. 6 tokens: [state, art, model, cost, 1000]
- b. 7 tokens: [state, of, the, art, model, cost, 1000]
- c. 8 tokens: [state, of, the, art, model, cost, 1, 000]
- d. 5 tokens: [state-of-the-art, model, cost, \$1,000]

Câu 19: Tại sao việc xóa bỏ toàn bộ Stopwords (ví dụ: “not”, “no”, “without”) đôi khi làm giảm nghiêm trọng chất lượng tìm kiếm đối với các truy vấn phủ định hoặc truy vấn ngắn?

- a. Làm cho trọng số IDF của tất cả các từ còn lại trong từ điển giảm xuống bằng 0.
- b. Làm mất hoàn toàn ngữ cảnh phủ định, biến truy vấn “not dangerous” thành “dangerous” làm hệ thống trả về kết quả ngược lại.
- c. Làm ma trận TF-IDF bị quá đặc (dense matrix) khiến không tính được Cosine.
- d. Làm tăng độ dài trung bình của các tài liệu trong tập corpus.

Câu 20: Do tất cả các phần tử trong ma trận đếm Count/TF-IDF đều không âm ($x_i \geq 0$), dải giá trị thực tế của Cosine Similarity giữa hai tài liệu luôn nằm trong khoảng nào?

- a. $[-\infty, +\infty]$
- b. $[0.0, 1.0]$
- c. $[-1.0, 1.0]$
- d. $[0.0, +\infty]$

Câu 21: Hạn chế lớn nhất nào của TF-IDF thúc đẩy sự ra đời của các phương pháp biểu diễn ngữ nghĩa như Word2Vec / Dense Embeddings?

- a. Tốc độ tính Cosine Similarity của TF-IDF quá chậm trên máy tính.
- b. TF-IDF không thể tính toán được trên tập dữ liệu lớn hơn 1.000 văn bản.
- c. Sự thiếu hụt khả năng biểu diễn mối quan hệ tương đồng ngữ nghĩa (Semantic Similarity) giữa các từ khác nhau về mặt chữ.
- d. TF-IDF bắt buộc phải dùng ngôn ngữ Python chứ không dùng được trên C++.

Câu 22: Mặc định trong lớp `TfidfVectorizer` của thư viện `scikit-learn`, tham số `norm` được đặt là ‘l2’. Việc này thực hiện thao tác toán học nào ngay sau khi tính xong bảng giá trị $TF \times IDF$?

- a. Giới hạn tất cả các giá trị TF-IDF nằm trong khoảng từ 0.0 đến 1.0 bằng hàm Sigmoid.

- b. Trừ đi giá trị trung bình của hàng và chia cho độ lệch chuẩn.
- c. Chia từng phần tử vector cho căn bậc hai của tổng bình phương tất cả các phần tử trong vector đó: $v' = \frac{v}{\|v\|_2}$.
- d. Chia từng phần tử cho tổng trị tuyệt đối của các phần tử trong vector.

Câu 23: Trong trường hợp hoàn hảo, tất cả Top-5 kết quả trả về của hệ thống tìm kiếm đều là tài liệu liên quan. Giá trị $P@5$ đạt được là bao nhiêu?

- a. 5.0
- b. 0.5
- c. 0.2
- d. 1.0 (hoặc 100%)

Câu 24: Thứ tự các bước chuẩn xác trong một Pipeline Tìm kiếm Văn bản dựa trên TF-IDF là:

- a. Text Preprocessing → Build TF-IDF Index → Query Vectorization → Cosine Similarity Computation → Sorting & Ranking Top-K
- b. Query Vectorization → Text Preprocessing → Cosine Similarity → Build TF-IDF Index
- c. Cosine Similarity → Text Preprocessing → Query Vectorization → Ranking
- d. Build TF-IDF Index → Text Preprocessing → Sorting → Cosine Similarity

Câu 25: Thuật toán Subword Tokenization (như Byte-Pair Encoding – BPE) loại bỏ được vấn đề OOV (Out-of-Vocabulary) nhờ cơ chế nào?

- a. Xóa bỏ các từ lạ khỏi câu truy vấn.
- b. Thêm từ lạ vào Google Search để lấy định nghĩa.
- c. Tự động dịch từ lạ sang tiếng Anh.
- d. Phân rã từ lạ thành các chuỗi ký tự con (subword units) hoặc ký tự đơn lẻ đã có sẵn trong từ điển.

Câu 26: Chỉ số MAP (Mean Average Precision) mở rộng Precision@K như thế nào khi đánh giá hệ thống tìm kiếm trên tập nhiều truy vấn?

- a. Đo tốc độ phản hồi tính bằng miligiây của hệ thống.
- b. Chỉ tính Precision duy nhất tại vị trí rank thứ 1.
- c. Lấy Precision@5 cộng với Recall@5 rồi chia đôi.
- d. Tính trung bình các giá trị Precision tại mỗi vị trí trả về tài liệu liên quan, sau đó lấy trung bình trên toàn bộ các truy vấn.

Câu 27: Thuật ngữ **Lexical Overlap** (Giao thoa từ vựng) trong hệ thống tìm kiếm TF-IDF ám chỉ điều gì?

- a. Sự xuất hiện trùng hợp của các chuỗi ký tự/từ giống hệt nhau giữa câu truy vấn và văn bản.
- b. Sự tương đồng ngữ nghĩa giữa các từ đồng nghĩa.
- c. Tỷ lệ các từ bị gõ sai chính tả trong tập dữ liệu.
- d. Việc đề dữ liệu trong bộ nhớ RAM khi chạy đa tiến trình.

Câu 28: Tại sao Khoảng cách Euclidean ($\|x - y\|_2$) không phù hợp để so sánh độ tương đồng giữa hai văn bản có độ dài lệch nhau nhiều nếu chưa qua chuẩn hóa vector?

- a. Do khoảng cách Euclidean chỉ dùng được cho không gian 2 chiều.
- b. Vì khoảng cách Euclidean luôn trả về giá trị âm.

- c. Khoảng cách Euclidean bị chi phối bởi độ dài tuyệt đối của vector, khiến hai văn bản cùng chủ đề nhưng một dài một ngắn có khoảng cách rất lớn.
- d. Vì khoảng cách Euclidean không thể tính được trên ma trận thưa.

Câu 29: Tại sao phải sử dụng $\text{set}(d)$ trong quá trình tính Document Frequency?

- a. Để loại bỏ các từ dài hơn 10 ký tự.
- b. Để đảm bảo mỗi từ chỉ được tính một lần trong mỗi tài liệu.
- c. Để sắp xếp từ theo thứ tự alphabet.
- d. Để chuyển từ thành số nguyên.

Câu 30: Tại sao việc xóa bỏ toàn bộ Stopwords như “not”, “no”, “without” có thể làm giảm chất lượng tìm kiếm?

- a. Làm cho IDF của tất cả các từ còn lại bằng 0.
 - b. Làm mất ngữ cảnh phủ định, ví dụ “not dangerous” trở thành “dangerous”.
 - c. Làm ma trận TF-IDF trở thành ma trận đặc.
 - d. Làm tăng độ dài trung bình của tài liệu.
-

PHẦN 2: ĐÁP ÁN VÀ LỜI GIẢI CHI TIẾT CHO TẤT CẢ CÁC CÂU HỎI

1. Bảng Tổng Hợp Đáp Án Nhanh

Câu	Đáp án	Câu	Đáp án	Câu	Đáp án
Câu 1	b	Câu 11	c	Câu 21	c
Câu 2	b	Câu 12	c	Câu 22	c
Câu 3	c	Câu 13	a	Câu 23	d
Câu 4	b	Câu 14	d	Câu 24	a
Câu 5	c	Câu 15	a	Câu 25	d
Câu 6	a	Câu 16	d	Câu 26	d
Câu 7	c	Câu 17	c	Câu 27	a
Câu 8	a	Câu 18	c	Câu 28	c
Câu 9	b	Câu 19	b	Câu 29	b
Câu 10	c	Câu 20	b	Câu 30	b

2. Lời Giải Thích Chi Tiết Từng Câu Hỏi

Câu 1: **Chọn b.** Khi mở rộng sang Bigram (1, 2), tập từ điển phải chứa thêm tất cả các cặp hai từ liên kề xuất hiện trong kho văn bản, khiến kích thước không gian vector $|V|$ tăng theo cấp số nhân. Ma trận biểu diễn thưa hơn nhiều vì một văn bản chỉ chứa tỷ lệ rất nhỏ các cặp từ này, nhưng giúp phân biệt được cụm từ có nghĩa (ví dụ “data science” khác với “data” và “science” rời rạc).

Câu 2: **Chọn b.** Trong tiếng Việt, từ ghép đóng vai trò biểu đạt một khái niệm độc lập. Nếu tách theo khoảng trắng thành hai token riêng lẻ “bệnh” và “nhân”, hệ thống mất hoàn toàn khái niệm y học “bệnh nhân” và gộp vào các tài liệu chỉ chứa từ “nhân” (người/nhân sự/hạt) hoặc từ “bệnh” độc lập, gây nhiễu rất lớn.

Câu 3: **Chọn c.** Áp dụng công thức tính Recall tại ngưỡng $K = 5$:

$$\text{Recall@5} = \frac{\text{Số tài liệu liên quan trong Top-5}}{\text{Tổng số tài liệu thực sự liên quan}} = \frac{2}{4} = 0.50 \text{ (50\%)}$$

Câu 4: **Chọn b.** Sắp xếp theo thứ tự từ điển (alphabetical order) là chuẩn thiết kế kỹ thuật của Scikit-Learn nhằm đảm bảo tính tiên định (deterministic): mỗi từ luôn tương ứng với cùng một chỉ số cột ma trận qua nhiều lần chạy, đồng thời khớp đúng cột khi thực hiện phép biến đổi `transform()` trên tập dữ liệu mới.

Câu 5: **Chọn c.** Với 30.000 văn bản và từ điển hàng chục nghìn chiều, hầu hết các ô đều mang giá trị 0 ($\geq 99\%$). Định dạng CSR (*Compressed Sparse Row*) nén dữ liệu bằng cách chỉ lưu trữ giá trị khác 0 kèm theo mảng chỉ số, giúp tiết kiệm bộ nhớ RAM gấp hàng chục lần và tối ưu hóa tốc độ nhân ma trận thưa theo hàng.

Câu 6: **Chọn a.** TF-IDF dựa trên so khớp chuỗi ký tự bề mặt (Lexical Matching). Do “COVID-19” khác mặt chữ với “coronavirus”, và “vaccine” khác hình thái với “vaccination” (chưa được đưa về gốc từ thông qua Lemmatization/Stemming), hệ thống không tìm thấy từ khóa chung nào giữa câu truy vấn và văn bản D .

Câu 7: **Chọn c.** Mean Reciprocal Rank (MRR) tính trung bình nghịch đảo vị trí của tài liệu liên quan đầu tiên:

$$\text{MRR} = \frac{1}{|Q|} \sum_{i=1}^{|Q|} \frac{1}{\text{rank}_i} = \frac{1}{3} \left(\frac{1}{1} + 0 + \frac{1}{4} \right) = \frac{1}{3} \times \frac{5}{4} = \frac{5}{12} \approx 0.417.$$

Câu 8: **Chọn a.** Ta thấy vector $y = 2x$. Hai vector cùng phương và cùng chiều trong không gian Euclid tạo với nhau một góc 0° , do đó $\cos(x, y) = \cos(0^\circ) = 1.0$.

Câu 9: **Chọn b.** $df(t)$ (Document Frequency) biểu thị số lượng tài liệu có chứa từ t . Sử dụng cấu trúc `set(d)` loại bỏ toàn bộ các token trùng lặp trong nội bộ văn bản d , đảm bảo mỗi tài liệu chỉ làm tăng biến đếm của từ đó đúng 1 đơn vị.

Câu 10: **Chọn c.** Recall luôn đo lường mức độ bao phủ trên toàn bộ tập đáp án chuẩn, do đó mẫu số bắt buộc là tổng số tài liệu thực sự liên quan có trong toàn bộ cơ sở dữ liệu (Ground-truth relevant documents).

Câu 11: **Chọn c.** Theo định nghĩa tần suất tương đối:

$$tf(t, d) = \frac{f(t, d)}{\sum_{t' \in d} f(t', d)}.$$

Nếu tài liệu d chỉ gồm đúng một từ lặp lại nhiều lần, số lần xuất hiện của từ đó bằng đúng tổng số từ của tài liệu, dẫn đến $tf(t, d) = 1.0$.

Câu 12: **Chọn c.** Áp dụng công thức Cosine Similarity:

$$\begin{aligned} x \cdot y &= (2)(1) + (0)(3) + (1)(0) + (2)(1) = 2 + 0 + 0 + 2 = 4, \\ \|x\|_2 &= \sqrt{2^2 + 0^2 + 1^2 + 2^2} = \sqrt{9} = 3, \\ \|y\|_2 &= \sqrt{1^2 + 3^2 + 0^2 + 1^2} = \sqrt{11} \approx 3.3166, \\ \cos(x, y) &= \frac{4}{3 \times \sqrt{11}} = \frac{4}{9.9499} \approx 0.402. \end{aligned}$$

Câu 13: **Chọn a.** Khi đặt `smooth_idf=False`, công thức của Scikit-Learn là $idf(t) = \log\left(\frac{N}{df(t)}\right) + 1$, trong đó hằng số $+1$ được cộng thêm vào ngoài logarit để giữ cho các từ xuất hiện ở tất cả các tài liệu ($df = N$) có $idf = 1$ thay vì bị triệt tiêu về 0.

Câu 14: **Chọn d.** Việc cộng 1 vào cả tử và mẫu trong công thức Smoothing IDF giúp tránh phép chia cho 0 khi gặp từ vựng mới chưa từng xuất hiện trong tập huấn luyện, đồng thời đảm bảo giá trị trong logarit luôn lớn hơn hoặc bằng 1, giúp $idf \geq 1$.

Câu 15: **Chọn a.** Khi tăng K , danh sách trả về dài hơn nên khả năng bao quát thêm tài liệu liên quan tăng lên (Recall@K tăng hoặc giữ nguyên). Ngược lại, việc lấy thêm nhiều kết quả ở các vị trí xếp hạng thấp hơn thường kéo theo nhiều tài liệu rác, làm tỷ lệ chính xác Precision@K có xu hướng giảm.

Câu 16: **Chọn d.** Đại lượng TF-IDF là tích của mức độ phổ biến cục bộ trong tài liệu (TF) và mức độ đặc trưng/hiếm gặp trên toàn bộ kho ngữ liệu (IDF), phản ánh chính xác tầm quan trọng tương đối của từ khóa đối với tài liệu đó.

Câu 17: **Chọn c.** Stemming cắt bỏ đuôi từ dựa trên các tập luật mẫu (heuristics) nên thực thi rất nhanh nhưng dễ sinh ra các chuỗi vô nghĩa; Lemmatization dùng từ điển hình thái và phân tích ngữ pháp để đưa từ về dạng nguyên thể (lemma) chuẩn xác hơn, đổi lại chi phí tính toán cao hơn.

Câu 18: **Chọn c.** Chuỗi đầu vào: `state-of-the-art model cost $1,000`. Khi thay thế toàn bộ ký tự không phải chữ/số (`-`, `$`, `,`) thành khoảng trắng, chuỗi trở thành: `state of the art model cost 1 000`. Tách theo khoảng trắng thu được đúng 8 token: `[state, of, the, art, model, cost, 1, 000]`.

Câu 19: **Chọn b.** Các từ dừng phủ định (negation stopwords) như “not”, “no” mang tải trọng ngữ nghĩa cực kỳ quan trọng. Loại bỏ chúng sẽ biến “not dangerous” thành “dangerous”, làm hệ thống trả về các tài liệu mang ý nghĩa hoàn toàn trái ngược với mong muốn của người dùng.

Câu 20: **Chọn b.** Vì ma trận tần suất và trọng số TF-IDF không bao giờ chứa giá trị âm ($x_i \geq 0$), hai vector luôn nằm trong góc phần tư thứ nhất của không gian vector, dẫn đến tích vô hướng luôn không âm. Vì vậy, khoảng giá trị thực tế của Cosine Similarity chỉ nằm trong đoạn $[0.0, 1.0]$.

Câu 21: **Chọn c.** Điểm yếu cố hữu của TF-IDF là tính trực giao giữa các từ (từ đồng nghĩa như “ô tô” và “xe hơi” bị coi là hai chiều độc lập). Điều này thúc đẩy sự ra đời của các mô hình vector dày (Dense Embeddings) giúp học được khoảng cách ngữ nghĩa thực tế.

Câu 22: **Chọn c.** Mặc định chuẩn hóa L_2 lấy từng phần tử chia cho chuẩn Euclid của cả vector:

$$\|v\|_2 = \sqrt{\sum_{i=1}^n v_i^2} \implies v' = \frac{v}{\|v\|_2}.$$

Quy trình này đưa độ dài của vector về 1.0, triệt tiêu sự thiên vị về điểm số đối với các văn bản có độ dài lớn.

Câu 23: **Chọn d.** Định nghĩa Precision tại Top-5:

$$P@5 = \frac{\text{Số tài liệu liên quan trong Top-5}}{5} = \frac{5}{5} = 1.0 \text{ (tương ứng 100\%).}$$

Câu 24: **Chọn a.** Thứ tự tuần tự chuẩn hóa: Tiền xử lý dữ liệu (Preprocess) $\rightarrow$ Xây dựng chỉ mục TF-IDF trên kho tài liệu $\rightarrow$ Vector hóa truy vấn $\rightarrow$ Tính độ tương đồng Cosine $\rightarrow$ Sắp xếp và trích xuất Top-K kết quả.

Câu 25: **Chọn d.** Thuật toán tách từ con (Subword Tokenization) chia các từ hiếm hoặc từ chưa từng thấy thành các đơn vị con (subwords) hoặc các ký tự đơn lẻ đã tồn tại trong vốn từ điển cơ sở, giải quyết hoàn toàn bài toán Out-Of-Vocabulary.

Câu 26: **Chọn d.** Chỉ số Mean Average Precision (MAP) tính điểm Average Precision (AP) cho từng truy vấn (trung bình các giá trị Precision tại từng vị trí xuất hiện tài liệu đúng), sau đó lấy trung bình cộng tất cả các điểm AP trên toàn bộ tập truy vấn thử nghiệm.

Câu 27: **Chọn a.** Lexical Overlap là mức độ trùng khớp bề mặt ký tự giữa các từ trong truy vấn và các từ trong văn bản, là cơ chế hoạt động cốt lõi của các thuật toán tìm kiếm truyền thống như TF-IDF và BM25.

Câu 28: **Chọn c.** Khoảng cách Euclid đo khoảng cách hình học tuyệt đối nối giữa hai điểm đầu mút vector. Nếu hai văn bản cùng nội dung nhưng một văn bản dài gấp 5 lần thì vector của nó sẽ dài hơn rất nhiều, dẫn đến khoảng cách Euclid rất lớn dù góc lệch định hướng (Cosine) bằng 0.

Câu 29: **Chọn b.** Áp dụng **set(d)** biến danh sách token thành một tập hợp chứa các phần tử duy nhất, đảm bảo tần suất xuất hiện của một từ trong tài liệu đó không bị đếm lặp vào giá trị Document Frequency (df).

Câu 30: **Chọn b.** Xóa các từ dừng phủ định làm sai lệch hoàn toàn ngữ nghĩa logic của câu văn, làm đảo ngược ý định tìm kiếm của người dùng (ví dụ: “not dangerous” bị biến đổi thành “dangerous”).